

Assignment — Phase 2 Submission

Fabulari — real-time MEAN-stack chat application

Name	Nicholas Bailey
Student number	s5393017
Workshop	Tuesday 6 October 2026, 3:00 pm
GitHub repository	https://github.com/Nick52553/3813ICT-Full-Stack-Chat-App
Documentation	docs/Phase2.md in the repository (copied in full below)

The following pages contain a verbatim copy of the raw Markdown document docs/Phase2.md from the repository above. The rendered version, including the storyboard screenshots in docs/screenshots/, can be viewed on GitHub.

Phase2.md — raw Markdown

Phase 2 – Nicholas Bailey – s5393017

Workshop: Tuesday 6 October 2026, 3:00 pm

Repository: <https://github.com/Nick52553/3813ICT-Full-Stack-Chat-App>

Contents

1. [Overview](#1-overview)
2. [Git Strategy](#2-git-strategy)
3. [Running the Project](#3-running-the-project)
4. [Specifications and Requirements](#4-specifications-and-requirements)
5. [System Architecture](#5-system-architecture)
6. [Data Structures (MongoDB)](#6-data-structures-mongodb)
7. [REST API](#7-rest-api)
8. [Socket.io API](#8-socketio-api)
9. [Angular Architecture](#9-angular-architecture)
10. [Design Documents](#10-design-documents)
11. [Testing](#11-testing)
12. [Known Limitations and Future Work](#12-known-limitations-and-future-work)

1. Overview

Fabulari is a full-stack, real-time chat application built on the MEAN stack (MongoDB, Express, Angular, Node.js) with Socket.io. Users are organised into **groups**, each containing **channels**, under three permission levels: **Super Admin**, **Group Admin** and **User**.

Phase 1 delivered the UI for every role, backed by JSON files. Phase 2 completes the application:

Area	Phase 2 implementation
Storage	All data in MongoDB using the official <code>mongodb` Node.js driver (no Mongoose), in seven collections</code>
Real-time chat	Socket.io: live messages, join/leave notices, online users, typing indicator, live deletion
Images	Profile pictures and image messages (PNG/JPG/GIF, max 2MB), validated by file content
Security	bcrypt password hashing, NoSQL-injection protection, server-side role checks, no self-promotion to admin
Validation	One set of rules shared by the server and the Angular forms, with per-field error messages
Testing	229 automated tests: 101 backend (Mocha/Chai), 104 Angular unit (Vitest), 24 end-to-end (Cypress)

2. Git Strategy

- **Single trunk branch.** Development was done on `master``, kept in a working state at every commit. As a sole developer working in small increments, short-lived feature branches added overhead without benefit, so each feature was built and committed directly in small steps.
- **Small, frequent commits.** Each Phase 2 feature was delivered as a sequence of self-contained commits (e.g. moving users to MongoDB, then groups, then channels, then requests), so the history shows how the work progressed and any step can be reverted on its own.
- **Tests before commit.** From the testing stage onward, the backend (`npm test``) and Angular (`ng test``) suites were run before committing.
- **`.gitignore``** excludes `node_modules/``, `dist/``, `.angular/``, `.env``, OS files, `backend/uploads/`` (user-uploaded images created at runtime) and Cypress output (`cypress/screenshots/``, `cypress/videos/``).
- The teaching staff member is added as a collaborator on the repository before marking.

3. Running the Project

Prerequisites

- Node.js 20+ (developed on Node 24)
- MongoDB running locally on `mongodb://127.0.0.1:27017``
- Angular CLI (`npm install -g @angular/cli``) – optional, `npx ng`` also works

Install and start

```
```bash
Backend (port 3000)
cd backend
npm install
npm run seed # load the Phase 1 data into MongoDB (WARNING: replaces existing data)
npm start

Frontend (port 4200) - in a second terminal
cd frontend/chat-frontend
npm install
npm start # then open http://localhost:4200
```
```

Seeded accounts (from the Phase 1 data – passwords are stored hashed in MongoDB):

| Username | Password | Role |
|-----------------|----------|-------------|
| supernick | nick1234 | Super Admin |
| ben@ben.com | ben123 | Group Admin |
| bobby@bobby.com | bobby123 | User |

These older passwords pre-date the Phase 2 password rules; they still work, but new accounts must meet the rules in [Section 4.3](#43-validation-rules).

If the database is empty, the app opens the **Register** page to create the first Super Admin.

Configuration

| Variable | Default | Purpose |
|------------|---------------------------|----------------------------------|
| MONGO_URL | mongodb://127.0.0.1:27017 | MongoDB server |
| DB_NAME | chatapp | Database name |
| UPLOAD_DIR | backend/uploads | Where uploaded images are stored |

Running the tests

See [Section 11.2](#112-how-to-run-the-tests).

4. Specifications and Requirements

4.1 Functional requirements

| ID | Requirement | Status | Where |
|------|---|--------|---|
| FR1 | Log in with username and password; feedback on failure | Done | <code>POST /api/login</code> , <code>Login</code> component |
| FR2 | First run creates the single Super Admin account | Done | <code>POST /api/bootstrap</code> , <code>Register</code> , <code>registerGuard</code> / <code>loginGuard</code> |
| FR3 | Anyone can sign up as a regular user | Done | <code>POST /api/users</code> , <code>Signup</code> |
| FR4 | Super Admin creates users with any role; only the Super Admin can create admin accounts | Done | <code>POST /api/users</code> (<code>requesterId</code> checked) |
| FR5 | Super Admin deletes users; they are removed from every group and channel | Done | <code>DELETE /api/users/:id</code> |
| FR6 | Users request new groups; only the Super Admin approves | Done | Requests of type <code>group</code> |
| FR7 | Users request to join a group; groups can set a minimum age | Done | Requests of type <code>join</code> , <code>ageLimit</code> |
| FR8 | Users request new channels; the group's admins (or Super Admin) approve | Done | Requests of type <code>channel</code> |
| FR9 | Admins request a user's removal or ban from a group | Done | Requests of type <code>ban</code> / <code>groupRemoval</code> |
| FR10 | A group can never be left without an admin | Done | Checked when approving a removal |
| FR11 | Group/Super Admins add members, promote and demote group admins | Done | <code>/api/groups/:id/members</code> , <code>/admins</code> , <code>/admins/demote</code> |
| FR12 | Group/Super Admins create channels and add users to them | Done | <code>/api/channels</code> , <code>/api/channels/:id/members</code> |
| FR13 | Real-time text chat within a channel | Done | Socket.io <code>sendMessage</code> / <code>newMessage</code> |
| FR14 | Users see others joining and leaving a channel | Done | <code>userJoined</code> / <code>userLeft</code> |
| FR15 | Image messages and profile pictures (PNG/JPG/GIF, ≤ 2MB) | Done | <code>/api/channels/:id/images</code> , <code>/api/users/:id/avatar</code> |
| FR16 | Users delete their own messages; deletions disappear for everyone live | Done | <code>DELETE /api/messages/:id</code> , <code>messageDeleted</code> |
| FR17 | All data stored persistently in MongoDB | Done | Section 6 |

| FR18 | Every admin action is recorded in an audit log, viewable by the Super Admin | Done | `audit` collection, `GET /api/audit`, `AuditLog` |
 | FR19 | Pages restricted by role | Done | `authGuard`, `superAdminGuard`, `groupAdminGuard`, plus server-side checks |

****Additional real-time features:**** who is online in the channel, a typing indicator, and a LIVE / JOINING / RECONNECTING connection status with automatic rejoin after a dropped connection.

4.2 Non-functional requirements

| Area | Requirement and how it is met |
|----------------------|--|
| Security – passwords | Stored only as bcrypt hashes (cost factor 10). Accounts from before hashing are upgraded automatically when the server starts. |
| Security – login | Inputs must be plain strings, preventing NoSQL injection (e.g. `{ "\$ne": "" }`). Unknown users and wrong passwords get the same response and take the same time, so usernames can't be discovered. |
| Security – roles | The server, not just the UI, checks who may review requests, create admin accounts, delete users, change avatars, read channels and delete messages. |
| Security – uploads | Image type is detected from the file's first bytes, not its name; files get random names; size is limited to 2MB; invalid files are never written to disk. |
| Consistency | Request review uses a single atomic update, so two admins can't both review the same request. If an approval can't be applied, the request returns to *pending*. Socket events from one connection are processed in order. |
| Efficiency | Indexes on every lookup field (Section 6.2). Images are stored as files with only their URL in MongoDB. Chat history is fetched newest-first with a limit. |
| Validation | Identical rules on the client and server (Section 4.3); every form shows errors next to the field. |
| Fault tolerance | Friendly messages when the server is unreachable; buttons disabled while a request is in flight; malformed socket events are rejected without affecting the server. |
| Accessibility | Labelled form fields, `aria-invalid` and `aria-describedby` on errors, `role="alert"` for error messages, new chat messages announced to screen readers (`aria-live`), alt text on images. |

4.3 Validation rules

Defined once in `backend/validation.js` and mirrored in `frontend/chat-frontend/src/app/utils/validation.ts`.

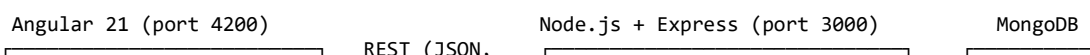
| Field | Rule |
|-----------------------------|---|
| Username | Required, 3-50 characters, letters/numbers and `@ . _ -` only, unique (case-insensitive) |
| Password (new accounts) | 8-100 characters, at least one uppercase letter |
| Age | Whole number, 13-120 |
| Date of birth (sign-up) | Required, not in the future, gives an age of 13-120 |
| Role | `user`, `groupAdmin` or `superAdmin` |
| Group / channel name | Required, 2-50 characters; group names unique, channel names unique within a group (case-insensitive) |
| Description, removal reason | Optional, up to 300 characters |
| Group minimum age | Optional, whole number 0-120 |
| Message | 1-2000 characters (or an image with an optional caption) |
| Image | PNG, JPG or GIF by content, maximum 2MB |

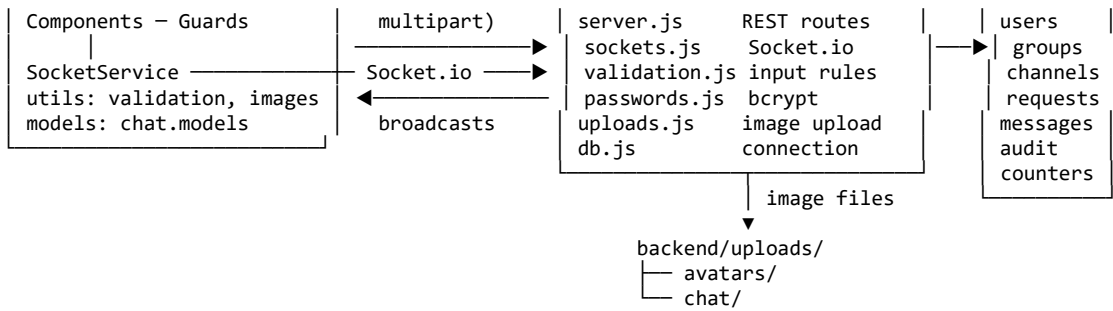
4.4 Changes since Phase 1

| Phase 1 plan | Phase 2 outcome | Reason |
|--|---|--|
| Mongoose schemas | Native `mongodb` driver | Mongoose is not permitted for this assignment |
| No request type for joining a group (Phase 1 Assumption 3) | `join` request type, with age-limit check | Lets users ask to join instead of relying on an admin to add them |
| Only the last 5 messages per channel kept on the server | All messages are stored; the history endpoint returns the newest 50 (up to 200) | Simpler and more useful; see [Section 12](#12-known-limitations-and-future-work) |
| Users may delete only their own messages | Group Admins and the Super Admin may also delete messages in their channels (logged in the audit log) | Needed for moderation |
| Super Admin does not take part in chat | Super Admin can read and moderate every channel | Needed for moderation |
| Planned services: `AuthService`, `UserService`, `GroupService`, etc. | `SocketService` implemented; other components still call `HttpClient` directly | See Section 12 |
| `DELETE` for groups and channels | Not implemented | Out of scope for the time available |

5. System Architecture

...





Backend files

| File | Responsibility |
|---------------|---|
| server.js | Express app, all REST routes, permission checks, starts the HTTP + Socket.io server |
| sockets.js | Socket.io events: channel rooms, presence, messages, typing |
| db.js | MongoDB connection, getNextId() (numeric ids), createIndexes() |
| validation.js | Input validation rules shared by all routes |
| passwords.js | bcrypt hashing and verification; upgrades old plain-text passwords |
| uploads.js | Image upload handling (multer), file-type detection, saving and deleting files |
| seed.js | npm run seed: loads the Phase 1 JSON data into MongoDB |
| e2e-server.js | Starts the server on the chatapp_e2e database for Cypress |

REST is used for data and file uploads; Socket.io is used for anything that must appear instantly for other users. Messages sent over either path are validated and saved by the same function, then broadcast over the socket.

6. Data Structures (MongoDB)

6.1 Collections

Every document has a numeric `id` (the Angular app and URLs use these), alongside MongoDB's own `_id`, which is never sent to the client. Ids come from the `counters` collection using an atomic `$inc`, so two simultaneous requests can never get the same id.

```

**`users`**
```js
{
 id: Number,
 username: String, // unique, case-insensitive
 password: String, // bcrypt hash, e.g. "$2b$10$..." - never sent to the client
 age: Number,
 role: "superAdmin" | "groupAdmin" | "user",
 avatarUrl: String // optional, e.g. "/uploads/avatars/<uuid>.png"
}

```

```

`groups`
```js
{
  id: Number,
  name: String,               // unique, case-insensitive
  description: String,
  ageLimit: Number,           // minimum age to join; 0 = none
  adminIds: [Number],         // user ids
  memberIds: [Number]         // user ids (admins are always members)
}

```

```

**`channels`**
```js
{
 id: Number,
 groupId: Number, // the group it belongs to
 name: String, // unique within its group, case-insensitive
 description: String,
 memberIds: [Number] // user ids
}

```

```

`messages`

```

```

```js
{
  id: Number,
  channelId: Number,
  groupId: Number,          // copied from the channel, for permission checks
  userId: Number,           // sender
  username: String,         // copied so history still shows a name if the user is deleted
  text: String,             // message, or optional caption for an image
  imageUrl: String | null,  // e.g. "/uploads/chat/<uuid>.gif"
  timestamp: String         // ISO 8601
}
```

```requests```
```js
{
 id: Number,
 type: "group" | "join" | "channel" | "ban" | "groupRemoval",
 requesterId: Number,
 groupId: Number | null, // all types except "group"
 targetUserId: Number | null, // "ban" and "groupRemoval"
 name: String, // proposed group/channel name
 description: String,
 ageLimit: Number, // "group" only
 reason: String,
 status: "pending" | "approved" | "denied",
 createdAt: String, // ISO 8601
 reviewedAt: String | null,
 reviewedBy: Number | null // reviewer's user id
}
```

```audit```
```js
{
  id: Number,
  action: "user.bootstraped" | "user.created" | "user.deleted"
        | "group.admin.promoted" | "group.admin.demoted"
        | "request.approved" | "request.denied" | "message.deleted",
  actorId: Number | null,   // who did it
  details: Object,         // e.g. { requestId, requestType, groupId }
  timestamp: String
}
```

```counters```
```js
{ _id: "users" | "groups" | "channels" | "requests" | "audit" | "messages", seq: Number }
```

```

6.2 Indexes

Created by `createIndexes()` in `db.js` (used by the seed script and the tests).

| Collection | Index | Purpose |
|------------|--------------------------------|--|
| all | { id: 1 } | unique Fast lookup by id; no duplicate ids |
| `users` | { username: 1 } | unique, case-insensitive Duplicate usernames rejected by the database itself |
| `groups` | { name: 1 } | unique, case-insensitive Duplicate group names rejected |
| `channels` | { groupId: 1, name: 1 } | unique, case-insensitive Unique names per group; also serves "channels in group X" |
| `requests` | { status: 1 } | Pending-request lists |
| `messages` | { channelId: 1, timestamp: 1 } | "Latest messages in channel X" without scanning |

6.3 Design decisions

- **Membership as id arrays** (`memberIds`, `adminIds`) inside groups and channels. Updates use `\$addToSet` and `\$pull`, which are atomic and never create duplicates, and checks such as "is user 3 an admin anywhere?" are a single query ({ adminIds: 3 }).
- **Uniqueness enforced by indexes**, not by "check then insert", so two simultaneous sign-ups with the same username can't both succeed.
- **Images on disk, URLs in MongoDB.** Keeps documents small and lets Express serve files directly. Files are deleted when the avatar is replaced or removed, when the message is deleted, or when the user is deleted.
- **Sender name copied into each message**, so history still makes sense after a user is deleted.

7. REST API

Base URL `http://localhost:3000`. Requests and responses are JSON, except the image uploads (`multipart/form-data`). Errors always have the form `{ "message": "..." }`.

| Status | Meaning |
|-----------|--|
| 200 / 201 | OK / created |
| 400 | Invalid input (the message explains which rule failed) |
| 401 | Wrong username or password |
| 403 | Not allowed for this user |
| 404 | User, group, channel, request or message not found |
| 409 | Duplicate (username, group name, channel name) or already a member |
| 500 | Unexpected server error |

****Identifying the user:**** the client sends the acting user's id (`requesterId`, `reviewerId` or `userId`, as listed below), and the server looks up that user's role before allowing the action. See [Section 12](#12-known-limitations-and-future-work).

7.1 Set-up and login

| Method | Endpoint | Body / query | Response | Errors |
|--------|-------------------------|-----------------------------|---|---|
| GET | `/` | — | { message, status: "online" } | — |
| GET | `/api/bootstrap-status` | — | { needsBootstrap: Boolean } (true when no Super Admin exists) | — |
| POST | `/api/bootstrap` | { username, password, age } | 201: the new Super Admin (user object) | 400 invalid, 403 already set up, 409 username taken |
| POST | `/api/login` | { username, password } | User object { id, username, age, role, avatarUrl } | 400 missing/non-string fields, 401 wrong username or password |

7.2 Users

| Method | Endpoint | Body / query | Who | Response | Errors |
|--------|-----------------------------|--|--|--|---|
| GET | `/api/users` | — | Anyone | Array of user objects (no passwords) | — |
| POST | `/api/users` | { username, password, age, role?, requesterId? } | Anyone for `role: "user"`; Super Admin (`requesterId`) for other roles | 201: user object | 400 invalid, 403 not Super Admin, 409 username taken |
| DELETE | `/api/users/:userId` | { requesterId= } | Super Admin | { message } | user removed from all groups and channels, avatar file deleted 403, 404 |
| POST | `/api/users/:userId/avatar` | multipart: `avatar` (file), `requesterId` | The user, or Super Admin | User object with new `avatarUrl`; old file deleted | 400 bad/missing/too-large file, 403, 404 |
| DELETE | `/api/users/:userId/avatar` | { requesterId= } | The user, or Super Admin | User object with `avatarUrl: null` | 403, 404 |
| GET | `/uploads/...` | — | Anyone | The image file | 404 |

7.3 Groups

| Method | Endpoint | Body / query | Response | Errors |
|--------|--------------------------------------|--|---|-----------------------------|
| GET | `/api/groups` | — | Array of groups, by id | — |
| GET | `/api/groups/:groupId` | — | Group object | 404 |
| POST | `/api/groups` | { name, description?, ageLimit?, adminIds?, memberIds? } | 201: group object | 400 invalid, 409 name taken |
| POST | `/api/groups/:groupId/members` | { userId } | Updated group (user added once) | 404 group or user |
| POST | `/api/groups/:groupId/admins` | { userId } | Updated group; user also made a member; a `user` becomes a `groupAdmin` | 404 |
| POST | `/api/groups/:groupId/admins/demote` | { userId } | Updated group; role reset to `user` if they no longer admin any group | 404 |

7.4 Channels

| Method | Endpoint | Body / query | Response | Errors |
|--------|------------------------------------|---|-----------------------------------|--|
| GET | `/api/channels` | — | Array of all channels | — |
| GET | `/api/groups/:groupId/channels` | — | Channels in the group | — |
| POST | `/api/channels` | { groupId, name, description?, memberIds? } | 201: channel object | 400 invalid / no group, 404 group, 409 name taken in group |
| POST | `/api/channels/:channelId/members` | { userId } | Updated channel (user added once) | 404 channel or user |

7.5 Requests

| Method | Endpoint | Body / query | Who | Response | Errors |
|--------|--------------------------|---|---|---|---|
| GET | /api/requests | ?status= ` ` ?requesterId= ` ` ?reviewerId= ` (all optional)                            | Anyone                                                                                                                | Matching requests. With `reviewerId`, only requests that user may review | 404 reviewer |
| POST | /api/requests | { type, requesterId, groupId?, targetUserId?, name?, description?, ageLimit?, reason? } | Anyone | 201: request with `status: "pending"` 400 invalid / unknown type, 403 under the group's age limit, 404 user/group, 409 already a member | |
| PUT | /api/requests/:requestId | { status: "approved" \ "denied", reviewerId } | Super Admin (any); Group Admin (their group's `join`, `channel`, `ban`, `groupRemoval`); only Super Admin for `group` | { message, request } | on approval the change is applied 400 bad status / already reviewed / would remove the last admin, 403, 404, 409 duplicate name |

What approval does:

| Type | Fields used | Effect when approved |
|--------------------|-------------------------------|---|
| group | name, description, ageLimit | Creates the group with the requester as its first member |
| join | groupId | Adds the requester to the group |
| channel | groupId, name, description | Creates the channel; all group members get access |
| ban / groupRemoval | groupId, targetUserId, reason | Removes the user from the group, its admins and its channels (blocked if they are the only admin) |

7.6 Messages

| Method | Endpoint | Body / query | Who | Response | Errors |
|--------|-----------------------------------|--|--|---|--|
| GET | /api/channels/:channelId/messages | ?userId= (required), ?limit= (default 50, max 200) | Channel members, the group's admins, Super Admin | Newest *limit* messages, oldest first | 403, 404 |
| POST | /api/channels/:channelId/messages | { userId, text } | Same | 201: message; also broadcast as `newMessage` | 400 empty / too long, 403, 404 |
| POST | /api/channels/:channelId/images | multipart: `image` (file), `userId`, `text?` (caption) | Same | 201: message with `imageUrl`; broadcast as `newMessage` | 400 bad/missing/too-large file or caption too long, 403, 404 |
| DELETE | /api/messages/:messageId | ?userId= | Sender, the group's admins, Super Admin | { message } | file deleted; broadcast as `messageDeleted`; logged if not the sender 403, 404 |

7.7 Audit log

| Method | Endpoint | Query | Who | Response | Errors |
|--------|------------|---------------|-------------|-----------------------------|--------|
| GET | /api/audit | ?requesterId= | Super Admin | Audit entries, newest first | 403 |

8. Socket.io API

The Socket.io server shares port 3000 with Express. Each channel is a room named `channel:<id>`. Events marked **ack** take a callback, which receives `{ ok: true, ... }` or `{ ok: false, message }`. Events from one connection are processed one at a time, in the order they arrived.

8.1 Client → server

| Event | Payload | Ack | Behaviour |
|--------------|-------------------------|-----------------|--|
| joinChannel | { userId, channelId } | { ok, users } | who is online Checks the user may access the channel, joins its room, announces `userJoined` to others (only for the user's first tab), broadcasts `onlineUsers` |
| leaveChannel | { channelId } | { ok } | Leaves the room; announces `userLeft` once the user's last tab has left |
| sendMessage | { channelId, text } | { ok, message } | Requires having joined the channel; re-checks access, validates and saves the message, broadcasts `newMessage` |
| typing | { channelId, isTyping } | - | Forwards `typing` to everyone else in the channel |

Closing the tab or losing the connection is handled like `leaveChannel` for every channel the socket was in. The Angular client automatically rejoins its channel after reconnecting.

8.2 Server → client

All are sent to the channel's room.

| Event | Payload | Sent when |
|----------------|------------------------------|---|
| newMessage | Message object (Section 6.1) | A message or image is sent, over the socket or REST – including to the sender |
| messageDeleted | { id, channelId } | A message is deleted |

```
`userJoined`	`{ channelId, userId, username, timestamp }`	A user opens the channel (not sent to that user)
`userLeft`	`{ channelId, userId, username, timestamp }`	A user's last connection leaves the channel
`onlineUsers`	`{ channelId, users: [{ id, username }] }`	Anyone joins or leaves
`typing`	`{ channelId, userId, username, isTyping }`	Another user starts or stops typing
```

8.3 Example

```
```js
socket.emit('joinChannel', { userId: 3, channelId: 1 }, ack => {
 // ack = { ok: true, users: [{ id: 3, username: 'bobby@bobby.com' }] }
});

socket.emit('sendMessage', { channelId: 1, text: 'Hello!' }, ack => {
 // ack = { ok: true, message: { id: 12, channelId: 1, text: 'Hello!', ... } }
});

socket.on('newMessage', message => { /* add to the chat */ });
```

---
```

9. Angular Architecture

Angular 21, standalone components, `HttpClient` for REST and `socket.io-client` for real-time events. The logged-in user is kept in `localStorage` under `currentUser`.

9.1 Components

| Component | Route | Purpose | Server calls |
|---------------------|-----------------------------|--|---|
| `Login` | `/login` | Username/password form with per-field errors and server feedback | `POST /api/login` |
| `Register` | `/register` | First-run creation of the Super Admin (only reachable while no users exist) | `POST /api/bootstrap` |
| `Signup` | `/signup` | Create an account; age is calculated from date of birth | `POST /api/users` |
| `Dashboard` | `/dashboard` | Landing page with links to groups, channels and profile | — |
| `Navbar` | (all signed-in pages) | Navigation; admin links shown by role; logout | — |
| `GroupList` | `/groups` | All groups; view channels or request to join | `GET /api/groups`, `POST /api/requests` |
| `ChannelList` | `/channels/:groupId` | Channels in a group, each linking to its chat | `GET /api/groups/:id`, `GET /api/groups/:id/channels` |
| `ChatWindow` | `/chat/:groupId/:channelId` | Real-time chat: history, sending text and images, join/leave notices, online users, typing, deleting | Messages REST routes, `GET /api/users` (avatars), Socket.io via `SocketService` |
| `Profile` | `/profile` | Account details; upload, change or remove the profile picture | `/api/users/:id/avatar` |
| `Requests` | `/requests` | Tabs to request a new group, join a group, a new channel, or a user's removal | `GET /api/groups`, `GET /api/users`, `POST /api/requests` |
| `GroupManagement` | `/groups/manage` | Create groups, add members, promote/demote admins, review join/channel/removal requests | Groups and requests routes |
| `ChannelManagement` | `/channels/manage` | Create channels, add users to channels | Channels routes |
| `UserManagement` | `/user-management` | Create users with a role, delete users, review new-group requests | Users and requests routes |
| `AuditLog` | `/audit-log` | The audit log, newest first | `GET /api/audit` |

9.2 Services

| Service | Purpose |
|--|--|
| `SocketService` (`services/socket.service.ts`) | The app's single Socket.io connection, opened on first use. `joinChannel`, `leaveChannel`, `sendMessage` (return the server's ack as a Promise), `sendTyping`, and `on(event)`, which returns an Observable that removes its listener on unsubscribe. Exposes a `connected` **signal** for the LIVE / RECONNECTING status and rejoins the current channel after a reconnect. |

9.3 Models (`models/chat.models.ts`)

| Interface | Shape |
|-----------------------|---|
| `ChatMessage` | { id, channelId, groupId, userId, username, text, imageUrl, timestamp } |
| `OnlineUser` | { id, username } |
| `PresenceEvent` | { channelId, userId, username, timestamp } |
| `OnlineUsersEvent` | { channelId, users: OnlineUser[] } |
| `TypingEvent` | { channelId, userId, username, isTyping } |
| `MessageDeletedEvent` | { id, channelId } |
| `SocketAck` | { ok, message?, users? } |
| `ChatItem` | A message or a system notice in the chat timeline |

Users, groups, channels and requests follow the document shapes in Section 6.1.

9.4 Shared utilities

| File | Contents |
|---------------------|--|
| utils/validation.ts | The validation rules (mirror of backend/validation.js), hasErrors(), and httpErrorMessage() (friendly message, including "Could not reach the server") |
| utils/images.ts | Allowed image types and size, validateImageFile(), imageSrc() (turns a stored path into a full URL) |

9.5 Routes and guards

| Path | Component | Guard |
|---------------------------|-------------------------|-----------------|
| '' | redirects to /login | - |
| /login | Login | loginGuard |
| /register | Register | registerGuard |
| /signup | Signup | - |
| /dashboard | Dashboard | authGuard |
| /groups | GroupList | authGuard |
| /groups/manage | GroupManagement | groupAdminGuard |
| /channels/:groupId | ChannelList | authGuard |
| /channels/manage | ChannelManagement | groupAdminGuard |
| /chat/:groupId/:channelId | ChatWindow | authGuard |
| /profile | Profile | authGuard |
| /user-management | UserManagement | superAdminGuard |
| /requests | Requests | authGuard |
| /audit-log | AuditLog | superAdminGuard |
| ** | redirects to /dashboard | - |

| Guard | Rule |
|-----------------|--|
| authGuard | Signed in, otherwise → /login |
| superAdminGuard | Super Admin only, otherwise → /dashboard (or /login if signed out) |
| groupAdminGuard | Super Admin or Group Admin, otherwise → /dashboard (or /login) |
| registerGuard | Only while no Super Admin exists, otherwise → /login |
| loginGuard | Sends visitors to /register until the Super Admin has been created |

Guards control what the UI shows; the server enforces the same permissions on every request.

10. Design Documents

10.1 Screens by role

| Screen | User | Group Admin | Super Admin |
|--|------|-------------|-------------|
| Login, Sign up | ✓ | ✓ | ✓ |
| Dashboard, Groups, Channels, Chat, Profile, Requests | ✓ | ✓ | ✓ |
| Group Management, Channel Management | | ✓ | ✓ |
| User Management, Audit Log | | | ✓ |
| Register (first run only) | | | ✓ |

10.2 Changes to the Phase 1 designs

- **Chat window** – now live:
 - The header shows who is online.
 - A status badge reads LIVE / JOINING... / RECONNECTING...
 - Join and leave notices appear in the message list.
 - A typing line sits above the composer.
 - The 📎 button opens a file picker with a preview and an optional caption.
 - Images show inline and open full size on click.
 - Profile pictures appear beside messages, falling back to the first letter of the name.
 - Delete appears on your own messages, and on all messages for the group's admins.
- **Profile** – shows the profile picture, with Add / Change / Remove buttons and success or error messages.
- **All forms** – errors appear under the field they belong to, the field is outlined in red, and buttons show progress ("Logging in...", "Creating...") and can't be double-clicked.
- **New screens** – Register (first run), Sign up, and Audit Log.

10.3 Responsive design

Grid pages (Dashboard, Group List, Channel List) use CSS Grid with auto-fit/auto-fill and minmax(), so columns collapse to a single column on narrow screens. Management pages stack

two-column form rows below 650px. The profile card stacks vertically below 480px, and chat images scale to the available width.

The navbar wraps when there are more links than fit (the Super Admin has eight). Below 700px, the logo takes its own row, the links wrap on to as many rows as needed, and the user and Logout sit underneath, so every link stays visible and tappable on a phone. Wide tables (the audit log) scroll sideways inside a focusable, labelled region instead of cutting columns off.

10.4 Storyboards

Screenshots of the running application at desktop (1280 × 800) and mobile (390 × 844) widths. They were captured automatically by ``npm run docs:screenshots`` (Section 11.2) from real data on the end-to-end test database, so they can be regenerated whenever the UI changes. Accounts shown: ``super`` (Super Admin), ``ben`` (Group Admin of `*Gaming*`) and ``bobby`` (User).

1. Login – failed attempt (all visitors)

The login card with a rejected password. The server's message appears below the form, and the fields keep their values so the user can correct them.

```
Desktop	Mobile
```

2. Register – first run only

Shown only while the database has no users; it creates the single Super Admin. Once that account exists, ``registerGuard`` redirects this page to login.

```
Desktop	Mobile
```

3. Sign up – validation errors

Submitting an invalid form: each problem appears under its own field, and the fields are outlined in red, before anything is sent to the server.

```
Desktop	Mobile
```

4. Dashboard (User)

Landing page after login, linking to groups, channels and the profile. The navbar shows only the links the user's role may use.

```
Desktop	Mobile
```

5. Groups (User)

Every group with its member and channel counts. `*Request to Join*` appears only on groups the user isn't in.

```
Desktop	Mobile
```

6. Channels in a group (User)

The channels of `*Gaming*`, each opening its chat.

```
Desktop	Mobile
```

7. Profile with picture (User)

The uploaded profile picture with **Change picture** / **Remove**, and a confirmation message.

```
Desktop	Mobile
```

8. Chat – live conversation (User)

Bobby (in the browser) chatting live with Ben (connected from a second client). It shows:

- the online-users list ("2 online: ben, bobby")
- the LIVE status
- profile pictures beside messages, with Ben's initial as the fallback
- an image message with a caption
- Delete on Bobby's own messages only
- Ben's typing indicator

```
Desktop	Mobile
```

9. Requests – new channel form (User)

The request tabs (New Group, Join Group, New Channel, Request Removal), with validation errors on the New Channel form.

```
Desktop	Mobile
```

10. Group Management – pending requests (Group Admin)

Ben's management page: create a group, add members, assign or demote admins, and his queue of pending **channel**, **join** and **removal** requests for Gaming, with Accept / Deny.

```
Desktop	Mobile
```

11. Channel Management (Group Admin)

Create a channel in a group and add users to channels.

```
Desktop	Mobile
```

12. User Management (Super Admin)

Create users with any role, review new-group requests (here "Chess Club"), and delete users. The full Super Admin navbar wraps on to extra lines rather than overflowing.

```
Desktop	Mobile
```

13. Audit Log (Super Admin)

Every recorded admin action, newest first, with readable times. On mobile the table scrolls sideways instead of cutting columns off.

```
Desktop	Mobile
```

11. Testing

11.1 Tools and methodology

| Level | Tool | What it tests |
|---------------------|---|---|
| Backend unit | Mocha + Chai | Pure functions: validation rules and password hashing |
| Backend integration | Mocha + Chai + Supertest | Every REST route against a real MongoDB test database: success, validation, permissions, error codes and data left behind |
| Backend sockets | Mocha + Chai + socket.io-client | Real clients connected to the real server on a random port: broadcasts, presence, ordering, robustness |
| Angular unit | Vitest (Angular test builder) + `HttpTestingController` | Components, guards and utilities; HTTP calls intercepted and answered by the test; sockets replaced with a fake service |
| End-to-end | Cypress | A real browser using the real app and server: login, sign-up, chat, images, permissions, request workflow |

Principles followed:

- **Isolated data.** Backend tests use `chatapp\_test` and Cypress uses `chatapp\_e2e`; both are reset to the same known data (`backend/test/fixtures.js`) before every test. A check refuses to reset any database whose name doesn't end in `\_test` or `\_e2e`, so real data can't be wiped.
- **Behaviour, not implementation.** Tests check what a user or client sees: responses, status codes, stored data, rendered text.
- **Unhappy paths included.** Invalid input, missing permissions, duplicates, oversized or fake files, unreachable server, malformed socket events, simultaneous reviews.
- **Tests verified to catch faults.** During development, rules were deliberately broken (e.g. removing the password uppercase rule, letting anyone review requests, letting logged-out users through the guard) to confirm the relevant tests then failed.
- **Real bug found by testing.** The Cypress chat test revealed that a message sent straight after opening a channel could be refused, because the send overtook the join on the server. Fixed by processing each connection's socket events in order; a backend regression test now covers it.
- **More bugs found by the storyboard run** (Section 10.4), each fixed and covered by a test:
 - **Channel Management was unreachable.** `/channels/manage` was matched by the `/channels/:groupId` route, which comes first. The route order is now fixed, with a Cypress regression test.
 - **The message box could wipe your next message.** It was cleared when the server replied, which erased anything typed while waiting. It now clears only if it still holds the sent text, with a unit regression test.

11.2 How to run the tests

MongoDB must be running. The tests never touch the real `chatapp` database.

```
```bash
Backend: 101 tests (about 12 seconds)
cd backend
npm test

Angular unit: 104 tests
cd frontend/chat-frontend
npm test -- --watch=false

End-to-end: 24 tests (ports 3000 and 4200 must be free - stop `npm start` first)
cd frontend/chat-frontend
npm run e2e
```
```

`npm run e2e` starts the backend on the `chatapp\_e2e` database and `ng serve`, runs Cypress headless, then stops both servers. To watch the tests run in the Cypress window instead, start `npm run e2e:backend` and `npm start` in two terminals, then run `npm run cy:open`.

`npm run docs:screenshots` (same folder, same free ports) regenerates the storyboard screenshots in `docs/screenshots/` (Section 10.4).

11.3 Summary

| Suite | Files | Tests |
|-----------------------|-----------|------------|
| Backend (Mocha) | 8 | 101 |
| Angular unit (Vitest) | 17 | 104 |
| End-to-end (Cypress) | 5 | 24 |
| Total | 30 | 229 |

11.4 All automated tests

Backend — `backend/`

test/auth.test.js (10)

| # | Suite | Test |
|----|------------------------------------|--|
| 1 | API: login | logs in with the right username and password |
| 2 | API: login | never returns the password or its hash |
| 3 | API: login | rejects a wrong password with 401 |
| 4 | API: login | gives the same 401 for an unknown user (no username probing) |
| 5 | API: login | rejects missing fields with 400 |
| 6 | API: login | blocks NoSQL injection via Mongo operators |
| 7 | API: bootstrap (first Super Admin) | reports that set-up is done when a Super Admin exists |
| 8 | API: bootstrap (first Super Admin) | refuses a second Super Admin with 403 |
| 9 | API: bootstrap (first Super Admin) | creates the first Super Admin on an empty system |
| 10 | API: bootstrap (first Super Admin) | validates bootstrap input |

test/groups-channels.test.js (14)

| # | Suite | Test |
|----|---------------|--|
| 1 | API: groups | lists groups in id order without Mongo _id |
| 2 | API: groups | gets one group, or 404 |
| 3 | API: groups | creates a group with trimmed name and numeric age limit |
| 4 | API: groups | rejects a duplicate group name regardless of case (409) |
| 5 | API: groups | validates group input (400) |
| 6 | API: groups | adds a member only once |
| 7 | API: groups | promoting an admin also makes them a member and a groupAdmin |
| 8 | API: groups | demoting the last group a user admin resets their role |
| 9 | API: groups | keeps the groupAdmin role while they still admin another group |
| 10 | API: channels | lists channels for one group |
| 11 | API: channels | creates a channel in an existing group |
| 12 | API: channels | allows the same channel name in different groups only |
| 13 | API: channels | rejects a missing group (400) or unknown group (404) |
| 14 | API: channels | adds a channel member only once, or 404 for unknown ids |

test/messages-images.test.js (17)

| # | Suite | Test |
|----|---------------------------|---|
| 1 | API: messages | saves a trimmed message with the sender name |
| 2 | API: messages | returns history oldest-first, limited to the newest N |
| 3 | API: messages | keeps non-members out of a channel (403) |
| 4 | API: messages | lets group admins and the Super Admin read without being members |
| 5 | API: messages | rejects empty and over-long messages (400) |
| 6 | API: messages > deleting | lets the sender delete their own message |
| 7 | API: messages > deleting | stops other members deleting someone else's message (403) |
| 8 | API: messages > deleting | lets a group admin moderate, and logs it in the audit log |
| 9 | API: images > avatars | uploads a PNG, stores its URL and serves the file |
| 10 | API: images > avatars | accepts JPG and GIF, and deletes the old file when replaced |
| 11 | API: images > avatars | checks the file contents, not the file name (400) |
| 12 | API: images > avatars | rejects files over 2MB (400) |
| 13 | API: images > avatars | only allows changing your own avatar, unless you're the Super Admin |
| 14 | API: images > avatars | removes an avatar and its file |
| 15 | API: images > chat images | saves an image message with an optional caption |
| 16 | API: images > chat images | rejects non-members without saving a file |
| 17 | API: images > chat images | deletes the file when its message is deleted |

test/passwords.test.js (5)

| # | Suite | Test |
|---|--|--|
| 1 | Unit: password hashing | produces a bcrypt hash, not the password |
| 2 | Unit: password hashing | salts each hash, so the same password hashes differently |
| 3 | Unit: password hashing | verifies the right password and rejects a wrong one |
| 4 | Unit: password hashing | never matches a plain-text or missing stored value |
| 5 | Unit: password hashing > hashPlainTextPasswords (start-up upgrade) | hashes old plain-text passwords once and leaves hashes alone |

test/requests.test.js (15)

| # | Suite | Test |
|---|-----------------------------------|---|
| 1 | API: requests > creating requests | saves a pending request with the next id |
| 2 | API: requests > creating requests | rejects unknown types and requesters |
| 3 | API: requests > creating requests | validates requests that will create something |

```

4	API: requests › creating requests	enforces a group age limit on join requests (403)
5	API: requests › creating requests	rejects joining a group you are already in (409)
6	API: requests › listing requests	filters by requester and status
7	API: requests › listing requests	shows each reviewer only what they may review
8	API: requests › reviewing requests	only the Super Admin can approve a new group
9	API: requests › reviewing requests	a group admin approves a channel for their own group
10	API: requests › reviewing requests	approving a join adds the user to the group
11	API: requests › reviewing requests	approving a ban removes the user from the group and its channels
12	API: requests › reviewing requests	will not remove the only admin of a group
13	API: requests › reviewing requests	a denied request changes nothing and cannot be reviewed again
14	API: requests › reviewing requests	puts a request back to pending if it cannot be applied
15	API: requests › reviewing requests	lets only one of two simultaneous reviews win

**`test/sockets.test.js`** (14)

#	Suite	Test
1	Sockets: real-time chat › joining and leaving	members can join and get the online list
2	Sockets: real-time chat › joining and leaving	non-members and unknown channels are refused
3	Sockets: real-time chat › joining and leaving	tells others when someone joins, and updates the online
list		
4	Sockets: real-time chat › joining and leaving	tells others when someone leaves or disconnects
5	Sockets: real-time chat › joining and leaving	a second tab for the same user does not announce
join/leave		
6	Sockets: real-time chat › messages	broadcasts a sent message to everyone in the channel, including
the sender		
7	Sockets: real-time chat › messages	handles a message sent straight after joining, without waiting
8	Sockets: real-time chat › messages	saves socket messages so they appear in the history
9	Sockets: real-time chat › messages	does not leak messages into other channels
10	Sockets: real-time chat › messages	refuses empty messages and channels you have not joined
11	Sockets: real-time chat › messages	broadcasts messages sent over REST, including images
12	Sockets: real-time chat › messages	broadcasts deletions
13	Sockets: real-time chat › typing and robustness	shows typing to others but not to the typist
14	Sockets: real-time chat › typing and robustness	survives malformed events without crashing

**`test/users.test.js`** (13)

#	Suite	Test
1	API: users › GET /api/users	lists every user without passwords
2	API: users › POST /api/users (sign-up)	creates a regular user with the next id
3	API: users › POST /api/users (sign-up)	stores only a bcrypt hash of the password
4	API: users › POST /api/users (sign-up)	rejects a duplicate username regardless of case (409)
5	API: users › POST /api/users (sign-up)	rejects invalid input with a helpful message (400)
6	API: users › POST /api/users (sign-up)	does not let anyone sign themselves up as an admin (403)
7	API: users › POST /api/users (sign-up)	lets the Super Admin create admin accounts
8	API: users › POST /api/users (sign-up)	records the new account in the audit log
9	API: users › DELETE /api/users/:id	only the Super Admin can delete users (403)
10	API: users › DELETE /api/users/:id	removes the user and their group/channel memberships
11	API: users › DELETE /api/users/:id	returns 404 for an unknown user
12	API: users › GET /api/audit	is only visible to the Super Admin
13	API: users › GET /api/audit	lists the newest entry first

**`test/validation.test.js`** (13)

#	Suite	Test
1	Unit: validation rules › validateUsername	accepts plain names and email addresses
2	Unit: validation rules › validateUsername	rejects missing, too short and too long names
3	Unit: validation rules › validateUsername	rejects spaces and symbols
4	Unit: validation rules › validateUsername	rejects non-string values such as Mongo operators
5	Unit: validation rules › validatePassword	accepts 8+ characters with an uppercase letter
6	Unit: validation rules › validatePassword	rejects short, lowercase-only and overly long passwords
7	Unit: validation rules › validateAge	accepts whole numbers 13-120, as numbers or numeric strings
8	Unit: validation rules › validateAge	rejects out-of-range, decimal and non-numeric ages
9	Unit: validation rules › other rules	validateRole only allows the three known roles
10	Unit: validation rules › other rules	validateName uses the given label
11	Unit: validation rules › other rules	descriptions and reasons are optional but limited to 300
characters		
12	Unit: validation rules › other rules	validateAgeLimit allows none, or 0-120
13	Unit: validation rules › other rules	firstError returns the first problem, or null

#### Angular unit – `frontend/chat-frontend/`

**`src/app/app.spec.ts`** (2)

```

```

#	Suite	Test
1	App	should create the app
2	App	renders the router outlet that hosts every page

**`src/app/components/audit-log/audit-log.spec.ts`** (2)

#	Suite	Test
1	AuditLog	asks for the log as the logged-in user and shows the entries
2	AuditLog	shows the server's message when access is refused

**`src/app/components/channel-list/channel-list.spec.ts`** (2)

#	Suite	Test
1	ChannelList	loads the group's name and channels from the URL's group id
2	ChannelList	links each channel to its chat window

**`src/app/components/channel-management/channel-management.spec.ts`** (6)

#	Suite	Test
1	ChannelManagement	loads groups, users and channels for the dropdowns
2	ChannelManagement	requires a group and a valid channel name
3	ChannelManagement	creates a trimmed channel in the chosen group
4	ChannelManagement	shows a duplicate-name error from the server
5	ChannelManagement	explains why Assign User does nothing without a selection
6	ChannelManagement	assigns the selected user to the selected channel

**`src/app/components/chat-window/chat-window.spec.ts`** (14)

#	Suite	Test
1	ChatWindow	should create
2	ChatWindow	adds a broadcast message to the timeline once
3	ChatWindow	ignores messages for other channels
4	ChatWindow	removes a message when the server says it was deleted
5	ChatWindow	shows join and leave notices
6	ChatWindow	shows who is typing
7	ChatWindow	updates the online user list
8	ChatWindow	sends trimmed text over the socket and clears the box
9	ChatWindow	rejects a non-image attachment before uploading
10	ChatWindow	uploads a selected image with its caption, then clears it
11	ChatWindow	marks the channel as joined once the server confirms
12	ChatWindow	does not send before the join is confirmed
13	ChatWindow	keeps what you typed while the previous message was still sending
14	ChatWindow	does not send an empty message

**`src/app/components/dashboard/dashboard.spec.ts`** (2)

#	Suite	Test
1	Dashboard	should create
2	Dashboard	includes the navigation bar

**`src/app/components/group-list/group-list.spec.ts`** (6)

#	Suite	Test
1	GroupList	loads groups from the server
2	GroupList	shows member and channel counts on each card
3	GroupList	knows which groups the user belongs to and admins
4	GroupList	sends a join request for a group the user is not in
5	GroupList	does not send a request for a group the user is already in
6	GroupList	shows the server's reason when a join is refused

**`src/app/components/group-management/group-management.spec.ts`** (8)

#	Suite	Test
1	GroupManagement	loads groups, users, and the requests a group admin handles
2	GroupManagement > creating a group	validates name, description and age limit first
3	GroupManagement > creating a group	sends a trimmed group and resets the form
4	GroupManagement > creating a group	shows a duplicate-name error from the server
5	GroupManagement > members and admins	asks for a group and a user before adding a member
6	GroupManagement > members and admins	adds the selected user to the selected group

```

```

| 7 | GroupManagement › members and admins | promotes and demotes admins through the right endpoints |
| 8 | GroupManagement | reviews a request as the logged-in admin |

**`src/app/components/login/login.spec.ts`** (5)

#	Suite	Test
1	Login	shows field errors and sends nothing when the form is empty
2	Login	logs in, remembers the user and opens the groups page
3	Login	shows the server's message for a wrong password
4	Login	explains when the server cannot be reached
5	Login	ignores a second click while logging in

**`src/app/components/navbar/navbar.spec.ts`** (4)

#	Suite	Test
1	Navbar	shows regular users only the everyday pages
2	Navbar	shows group admins the management pages but not user admin
3	Navbar	shows the Super Admin everything
4	Navbar	logout forgets the user and goes to the login page

**`src/app/components/profile/profile.spec.ts`** (6)

#	Suite	Test
1	Profile	should create
2	Profile	shows initials when there is no picture
3	Profile	rejects a file that is not an image without calling the server
4	Profile	rejects an image over 2MB without calling the server
5	Profile	uploads a valid image and saves the new picture
6	Profile	shows the server's message when an upload fails

**`src/app/components/register/register.spec.ts`** (3)

#	Suite	Test
1	Register (first Super Admin)	requires an age and a confirmed password
2	Register (first Super Admin)	creates the Super Admin and logs them in
3	Register (first Super Admin)	goes to login if someone else already finished set-up

**`src/app/components/requests/requests.spec.ts`** (8)

#	Suite	Test
1	Requests	only checks the fields on the current tab
2	Requests	switching tabs hides errors from the last attempt
3	Requests	a new-group request needs a valid name
4	Requests	sends a new-group request
5	Requests	sends a join request with a numeric group id
6	Requests	a channel request needs a group and a name
7	Requests	sends a removal request with an optional reason
8	Requests	shows the server's reason when a request is refused

**`src/app/components/signup/signup.spec.ts`** (6)

#	Suite	Test
1	Signup	flags every problem and sends nothing
2	Signup	rejects a birth date in the future
3	Signup	rejects anyone younger than 13
4	Signup	works out age correctly around a birthday
5	Signup	creates the account with the calculated age
6	Signup	shows a taken-username error from the server

**`src/app/components/user-management/user-management.spec.ts`** (6)

#	Suite	Test
1	UserManagement	loads users and only the pending new-group requests
2	UserManagement › creating a user	validates before sending anything
3	UserManagement › creating a user	sends the chosen role with the Super Admin as requester
4	UserManagement › creating a user	shows the server's error
5	UserManagement	deletes a user as the Super Admin
6	UserManagement	approves a new-group request and refreshes the list

**`src/app/guards.spec.ts`** (8)

```

```
#	Suite	Test
1	Route guards › authGuard	sends logged-out visitors to /login
2	Route guards › authGuard	lets logged-in users through
3	Route guards › role guards	superAdminGuard only admits the Super Admin
4	Route guards › role guards	groupAdminGuard admits group admins and the Super Admin
5	Route guards › role guards	sends regular users to the dashboard and visitors to login
6	Route guards › first-run guards	registerGuard only allows /register before set-up
7	Route guards › first-run guards	loginGuard sends visitors to /register before set-up
8	Route guards › first-run guards	fails safe when the server is unreachable
```

src/app/utils/validation.spec.ts (16)

```
#	Suite	Test
1	validation rules › validateUsername	accepts plain names and emails
2	validation rules › validateUsername	requires a value
3	validation rules › validateUsername	enforces 3-50 characters
4	validation rules › validateUsername	rejects spaces and symbols
5	validation rules › validatePassword	accepts 8+ characters with an uppercase letter
6	validation rules › validatePassword	rejects short, missing and all-lowercase passwords
7	validation rules › validatePasswordMatch	checks the confirmation matches
8	validation rules › validateAge	accepts whole numbers from 13 to 120
9	validation rules › validateAge	rejects out-of-range, fractional and empty values
10	validation rules › validateName	uses the label in its messages
11	validation rules › optional fields	allows an empty description but limits its length
12	validation rules › optional fields	allows no age limit, 0 to 120 otherwise
13	validation rules › helpers	validateRequired treats null and empty string as missing
14	validation rules › helpers	hasErrors spots any message
15	validation rules › helpers	httpErrorMessage explains an unreachable server
16	validation rules › helpers	httpErrorMessage prefers the server's message
```

End-to-end – frontend/chat-frontend/

cypress/e2e/auth.cy.js (7)

```
#	Suite	Test
1	Login and logout	sends logged-out visitors to the login page
2	Login and logout	shows field errors when the form is empty
3	Login and logout	rejects a wrong password with a clear message
4	Login and logout	logs in, shows the groups, and logs out again
5	Sign up	explains every problem and does not create the account
6	Sign up	creates an account and logs the new user in
7	Sign up	shows the server error for a taken username
```

cypress/e2e/chat.cy.js (5)

```
#	Suite	Test
1	Chat	reaches a channel through the groups and channels pages
2	Chat	sends a message that is saved and still there after reloading
3	Chat	shows another user joining, typing, chatting and leaving - live
4	Chat	deletes your own message for everyone, but not other people's
5	Chat	explains when you aren't a member of a channel
```

cypress/e2e/images.cy.js (3)

```
#	Suite	Test
1	Images	uploads a profile picture and shows it
2	Images	refuses a file that is not an image
3	Images	sends an image with a caption in chat
```

cypress/e2e/permissions.cy.js (6)

```
#	Suite	Test
1	Permissions by role	a regular user sees no admin links and is turned away from admin pages
2	Permissions by role	a group admin can manage groups but not users
3	Permissions by role	a group admin can open Channel Management from the navbar
4	Permissions by role	a regular user cannot open Channel Management
5	Permissions by role	the Super Admin can reach user management and the audit log
6	Permissions by role	the Super Admin creates a user, which is recorded in the audit log
```

cypress/e2e/requests.cy.js (3)

| # | Suite | Test |
|---|----------|---|
| 1 | Requests | blocks an underage user from asking to join an 18+ group |
| 2 | Requests | a user requests a channel and the group admin approves it |
| 3 | Requests | validates a request form before sending it |

12. Known Limitations and Future Work

| Limitation | Impact | Improvement |
|--|--------|-------------|
| <p>**No session tokens.** The client sends the acting user's id (<code>`requesterId`</code>, <code>`userId`</code>), and the server trusts it after checking that user's role. A user who edits their requests by hand could act as someone else. Issue a signed token (e.g. JWT) at login and identify the user from it on every REST call and socket connection. </p> <p>Some management routes (<code>`POST /api/groups`</code>, <code>`/members`</code>, <code>`/admins`</code>, <code>`/api/channels`</code>) check input but not the caller's role. Only the UI guards stop regular users calling them. Require a Group/Super Admin identity, as the request-review route already does. </p> <p>Components other than chat call <code>`HttpClient`</code> directly with a hard-coded server URL. Duplicated code; changing the server address means editing many files. Move calls into <code>`AuthService`</code>, <code>`GroupService`</code>, etc., with the URL in Angular environment config. </p> <p>All messages are kept; history returns the newest 50 (up to 200). Older messages can't be loaded from the UI. "Load older messages" paging using the existing <code>`limit`</code> and a <code>`before`</code> cursor. </p> <p>No delete for groups or channels. Must be removed directly in the database. <code>`DELETE`</code> routes that also clean up messages and memberships. </p> | | |